

A Real-Time AI-Based Exam Proctoring System

A PROJECT REPORT

Submitted in partial fulfilment of the
requirement for the award of the degree of

BACHELOR OF TECHNOLOGY (B. Tech)

in

Computer Science and Engineering

by

Rushan Hala

(Reg. No.: 229301300)

Under the supervision of

DR. TAPAN KUMAR DEY



**MANIPAL UNIVERSITY
JAIPUR**

(University under Section 2(f) of the UGC Act)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING,

SCHOOL OF CSE,

MANIPAL UNIVERSITY JAIPUR,

RAJASTHAN, INDIA – 303007

MAY, 2026

12/05/26

CERTIFICATE

This is to certify that the project entitled "A Real-Time AI-Based Exam Proctoring System" is a Bonafide work carried out as part of the course CS 4270, under my guidance from January 2026 to May 2026 by Rushan Hala 229301300, student of Computer Science and Engineering, 8th Semester at the Department of Computer Science and Engineering, Manipal University Jaipur, during the academic semester 8th in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, at MUJ, Jaipur.

Dr. Tapan Kumar Dey

Project Guide, Dept of Computer Science and Engineering

Manipal University Jaipur

Dr. Neha Chaudhary

HOD, Dept of Computer Science and Engineering

Manipal University Jaipur

STUDENT DECLARATION

I hereby declare that this project "A Real Time AI Based Exam Proctoring System" is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma of the University or other Institute, except where due acknowledgements have been made in the text

ACKNOWLEDGEMENTS

I want to express my sincere thanks to everyone who supported me during this project. First, I extend my gratitude to my project supervisor for their excellent guidance, encouragement, and constructive feedback while developing ProctorAI. Their knowledge in artificial intelligence and computer vision helped shape this work.

I am thankful to the Head of the Department of Computer Science and Engineering, Manipal University Jaipur, for providing the necessary resources and infrastructure that made this project possible. The laboratory facilities and computational resources from the department were crucial for successfully implementing this system.

I appreciate the Faculty of Science, Technology, and Architecture for creating an environment that promotes academic excellence and innovation. The courses in machine learning, computer vision, and software engineering I took during my B.Tech program provided the foundation for this project. I also acknowledge the help from the open-source community, especially the developers and maintainers of OpenCV, MediaPipe, Ultralytics YOLOv8, Firebase, and the Python ecosystem. Without these effective and freely available tools, completing a project of this scope would not have been possible in an academic setting.

I want to thank my classmates and peers who participated in testing and evaluating the ProctorAI system. Their honest feedback during trial examination sessions helped find edge cases and strengthen the violation detection pipeline.

Lastly, I am deeply grateful to my family for their constant support, patience, and encouragement throughout my academic journey. This work would not have happened without their love and understanding.

ABSTRACT

The quick growth of online learning and remote education, sped up by global events like the COVID-19 pandemic, has highlighted the importance of academic integrity in education. Traditional exam systems depend on physical supervision, which struggles with human attention, scalability, and consistency. The lack of reliable automated proctoring systems in schools leads to opportunities for cheating, which threatens the credibility of degrees from universities. ProctorAI fills this need by creating a real-time, artificial intelligence-based exam proctoring system that monitors student behavior through a standard webcam and browser, identifies various cheating signs, and reports violations to supervising faculty instantly.

The method used in this project combines several advanced computer vision and machine learning technologies into a single workflow. The system uses OpenCV for capturing and processing video frames in real time, Google MediaPipe for detecting facial landmarks and estimating head positions, a custom-trained YOLOv8 nano model for detecting prohibited devices, specifically mobile phones, and the sounddevice library for monitoring ambient audio levels. A WebSocket-based system connects the local proctoring client to a cloud backend hosted on Google Firebase Firestore and Cloudinary. A Chrome browser extension monitors digital behavior, like tab switching and window focus changes. The system detects six types of violations: prolonged mobile phone use for more than two seconds, the absence of the face from the camera frame for more than three seconds, the presence of multiple faces in the exam frame, sustained gaze deviation beyond set time limits, ambient audio detection indicating speech or whispering, and browser tab switching or unfocused window events. Each violation is logged with a timestamp, a JPEG screenshot uploaded to Cloudinary, and a structured record written to Firebase Firestore in real time. A dashboard for teachers, deployed on Netlify, provides live monitoring with per-student violation timelines, heatmap visualizations of violation patterns, screenshot viewing, and post-exam PDF report generation. The system automatically ends an exam session when it detects six cumulative violations.

The main tools and technologies used in this project include Python 3.10 with the Ultralytics, MediaPipe, OpenCV, and sounddevice libraries for the local proctoring client; websockets and firebase-admin Python packages for cloud connectivity; Google Firebase Firestore for real-time NoSQL database tasks; Cloudinary for scalable image storage; the Chrome Extension Manifest V3 API with JavaScript for browser-side monitoring; Firebase Authentication for secure login; and Netlify for hosting the teacher dashboard. The exam platform is designed as a single-page application built with HTML/CSS/JavaScript, featuring a Firebase-authenticated rules page, proctor verification gate, and a twenty-question general knowledge assessment module.

LIST OF TABLES

Table No.	Page No.
Table 2.1 – Technology Stack Summary	8
Table 2.2 – MediaPipe Face Mesh Landmark Indices Used	10
Table 3.1 – Violation Types and Detection Parameters	15
Table 3.2 – WebSocket Message Protocol	18
Table 4.1 – Firebase Firestore Collection Schema: students	22
Table 4.2 – Firebase Firestore Collection Schema: violations	23
Table 4.3 – Violation Severity Classification	24
Table 5.1 – Detection Accuracy Observations	28
Table 5.2 – System Latency Measurements	29

TABLE OF CONTENTS

Contents	Page No.
Acknowledgements	i
Abstract	ii
List of Tables	iii
List of Figures	iv
Chapter 1 INTRODUCTION	
1.1 Introduction and Motivation	1
1.2 Project Statement and Objectives	3
1.3 Organization of Report	4
Chapter 2 BACKGROUND MATERIAL	
2.1 Conceptual Overview	5
2.2 Technologies Involved	8
2.3 Literature Review and Related Work	13
Chapter 3 METHODOLOGY	
3.1 System Architecture	15
3.2 Detection Pipeline Design	16
3.3 Violation Handling and Temporal Filtering	18
3.4 Cloud Communication Architecture	19
3.5 Browser Extension Design	20
Chapter 4 IMPLEMENTATION	
4.1 Development Environment	21
4.2 Webcam Proctoring Module	21
4.3 WebSocket Server Module	23
4.4 Firebase Integration	23
4.5 Cloudinary Integration	24
4.6 Teacher Dashboard	25
4.7 Chrome Extension	26
4.8 Student Exam Platform	27
Chapter 5 RESULTS AND ANALYSIS	
5.1 Detection Performance	28
5.2 System Latency	29
5.3 Violation Evidence Samples	30
Chapter 6 CONCLUSIONS AND FUTURE SCOPE	
6.1 Conclusions	31
6.2 Future Scope of Work	32
References	34
Annexures	35

Chapter 1: INTRODUCTION

1.1 Introduction and Motivation

Maintaining the integrity of academic examinations is essential for a trustworthy educational system. When students earn degrees through dishonest means, the value of those credentials diminishes for employers, professional bodies, and society. Traditionally, this integrity has relied on human invigilators who oversee examination rooms for signs of cheating. While effective in smaller settings, this method has significant limitations in today's educational environment.

The global shift towards online and hybrid education, especially during the COVID-19 pandemic from 2020 to 2022, has shown that traditional invigilation methods are inadequate. When exams moved online, institutions struggled to prevent cheating. Students at home could easily use mobile phones, access external resources, switch browser windows, receive help from others nearby, or communicate with collaborators—all without an effective detection system.

Even in settings like computer labs, one invigilator cannot effectively monitor thirty or more students at once. Human attention is limited, and even a brief lapse in focus during a forty-five-minute exam creates an opportunity for dishonesty. There is also a wide range of invigilation standards across different sessions, examiners, and institutions.

Automated proctoring systems can effectively address these challenges. By using computer vision, machine learning, and real-time communication technologies, it's possible to create a system that continuously monitors student behavior throughout an exam, avoiding the fatigue and inconsistency of human monitoring. Such a system can supervise multiple students, keep an objective record of violations, and alert faculty in real-time with evidence after the exam.

ProctorAI was developed with these goals in mind. It monitors students via their laptop webcam and browser, detecting various cheating-related behaviors. These include phone use, leaving the exam seat, additional people in view, looking away from the screen, audible speech, and switching browser tabs. The system works in real-time, uploading violations to a cloud database and making them visible to faculty through a web-based dashboard.

The motivation for this project arises from three needs: the rise of online and computer-based exams, the limitations of human invigilation at scale, and the availability of sophisticated machine learning tools that make automated monitoring practical for academic projects.

1.2 Project Statement and Objectives

The project statement is: To design, implement, and evaluate a real-time AI-based examination proctoring system that uses computer vision, machine learning, and web technologies to detect and report cheating behaviors during online and computer-based examinations, capturing evidence to be shared with supervising faculty via a cloud-connected dashboard.

The specific objectives of the ProctorAI project are:

- To implement real-time face detection and head pose estimation using Google MediaPipe to determine when a student is not looking at the exam screen or has left their seat.
- To train and deploy a custom YOLOv8 object detection model to identify mobile phones in the webcam view, with temporal filtering to reduce false positives from brief interruptions or reflections.
- To implement ambient audio monitoring using the sounddevice library to recognize speech or other significant sounds that may signal communication with outside parties during the exam.

- To develop a Chrome browser extension that detects and reports tab switching, window focus changes, and attempts to open developer tools, ensuring comprehensive monitoring of digital exam behavior.
- To build a WebSocket-based communication setup that transmits violations and compressed screenshot evidence from the local proctoring client to a cloud backend in real-time and with minimal delay.
- To integrate with Firebase Firestore for real-time NoSQL data storage, Firebase Authentication for secure teacher access, and Cloudinary for scalable cloud storage of screenshot evidence.
- To create a teacher-facing web dashboard on Netlify that allows live monitoring of all examination sessions, provides violation timelines for each student with heatmap visuals, enables evidence review, and includes post-exam report generation.
- To design a student examination platform with a proctoring verification gate that checks if the Chrome extension is active before granting exam access, outlines clear rules, and allows a sample twenty-question general knowledge test.

1.3 Organization of Report

The rest of this report is divided into five chapters. Chapter 2 gives detailed background information on the concepts of computer vision, object detection, and web technologies relevant to the project, along with a review of the involved technologies and a discussion of related work in automated examination proctoring.

Chapter 3 describes the methodology used in the project, including system architecture, the design of the multi-modal detection pipeline, how violations are handled and filtered over time, the cloud communication architecture, and the design of the Chrome extension.

Chapter 4 covers the implementation in detail, including the development environment, the webcam proctoring module and its relation to the original code, the WebSocket server module, Firebase and Cloudinary integration, the teacher dashboard, the Chrome extension, and the student examination platform.

Chapter 5 presents the results and analysis of the system, including detection performance across violation categories, system latency measurements, and examples of violation evidence collected during testing.

Chapter 6 concludes with insights from the project and suggests future work that could enhance the capabilities, security, and usability of the ProctorAI system.

Chapter 2: BACKGROUND MATERIAL

2.1 Conceptual Overview

2.1.1 Computer Vision and Real-Time Video Processing

Computer vision is the field focused on helping computers understand and interpret visual information from images and videos. In examination proctoring, computer vision allows systems to analyze webcam frames in real time and identify behavioral signs from the visual data. ProctorAI uses OpenCV (Open Source Computer Vision Library) for video capture. OpenCV, originally created by Intel and now supported by a global community, offers the VideoCapture interface. This interface simplifies camera access and provides a continuous stream of frames as NumPy arrays. Each frame is a three-dimensional array with a shape of (height, width, 3). The three channels represent the Blue, Green, and Red (BGR) color components according to OpenCV's convention. The use of BGR instead of the more common RGB is a result of early camera hardware, and it's essential to consider this when using frames with other libraries that expect RGB input. To process frames in real time, the system must perform all calculations within the available time set by the target frame rate. At thirty frames per second, each frame gets about 33 milliseconds for processing. The most demanding operation in this process, YOLO object detection inference, takes between 30 and 100 milliseconds, depending on the hardware. If the inference time approaches or exceeds the frame budget, the processing rate falls below the camera's native frame rate. In practice, a rate of ten to fifteen frames per second is sufficient for detecting slower behavioral events relevant to examination proctoring, such as phone use, gaze deviation, and face absence, since these events last seconds instead of milliseconds.

2.1.2 Facial Landmark Detection and Head Pose Estimation

Facial landmark detection involves locating specific points on a human face within an image. These points, which include the corners of the eyes, the tip of the nose, the corners of the mouth, and the contour of the jaw, create a structured geometric representation of the face. This representation can be used for tasks like head pose estimation, gaze direction analysis, expression recognition, and face alignment.

Google MediaPipe offers the FaceMesh solution, which detects 468 three-dimensional facial landmarks in real time through a two-stage process. The first stage uses a lightweight face detector called BlazeFace to find faces in the image. In the second stage, a regression model predicts the exact x, y, z coordinates of all 468 landmarks for each detected face. When the `refine_landmarks` parameter is set to `True`, ten additional iris landmarks (indices 468 through 477) are estimated for more accurate gaze direction analysis.

Head pose estimation determines the orientation of the head in 3D space based on the 2D positions of facial landmarks in the image. ProctorAI uses a straightforward geometric method. It compares the horizontal position of the nose tip (landmark 1) to the midpoint between the eye corners (landmarks 33 and 263) to figure out if the student is looking left, right, or straight ahead. A threshold of 20 pixels is applied to prevent false detections caused by minor head movements. An additional vertical check looks at the y-coordinate of the nose tip in relation to the eye corners. If the nose is significantly lower than the eyes (with a threshold of 0.09 in normalized coordinates), the system classifies the student as looking downward, which may suggest they are reading notes underneath the desk.

2.1.3 Object Detection with Deep Neural Networks

Object detection is the task of identifying and locating objects of defined categories within an image at the same time. Unlike image classification, which assigns a single label to the whole image, object detection creates a set of bounding boxes, or rectangles defined by their top-left and bottom-right pixel

coordinates. Each bounding box comes with a class label and a confidence score that shows the model's certainty that the detected area contains the predicted object.

Modern object detectors that use deep learning fall into two main categories: two-stage detectors, which propose candidate object regions and then classify them, and one-stage detectors, which do both tasks in one go. One-stage detectors are usually faster and more suitable for real-time uses, but they might sacrifice accuracy for smaller or hidden objects.

The YOLO (You Only Look Once) model family, introduced by Redmon et al. in 2016, is the most commonly used one-stage detector. Its main concept divides the input image into a grid. For each grid cell, it predicts a fixed number of bounding boxes along with the likelihood that each box contains an object, as well as the probabilities of different classes given the presence of an object. Non-Maximum Suppression (NMS) is applied afterward to filter out duplicate overlapping predictions for the same object, keeping only the highest-confidence detection.

ProctorAI uses YOLOv8, the eighth version of the YOLO architecture created by Ultralytics. YOLOv8 includes a cleaner anchor-free detection head, better feature extraction, and an easier Python API. The nano variant (YOLOv8n) is the base model. It has the fewest parameters and lowest inference time while still providing enough accuracy for objects relevant to examination proctoring. A custom model (best.pt) was trained by fine-tuning YOLOv8n on a dataset specific to mobile phone images taken in exam-like settings. This dataset included partially visible phones, phones at various angles, and phones partially blocked by clothing or furniture.

2.1.4 Audio Signal Processing

ProctorAI implements audio monitoring using the sounddevice library, which provides Python bindings for the cross-platform PortAudio audio I/O library. The system captures microphone input as a continuous stream of samples and calculates the Root Mean Square (RMS) amplitude for each audio block. RMS measures the average signal power for audio and is calculated as the square root of the average of the squared sample values. In a silent room, RMS values are close to zero, while speech typically produces RMS values ranging from 0.05 to 0.20, depending on microphone sensitivity and speaker distance.

The system uses a block size of 1600 samples at a sample rate of 16,000 Hz, which corresponds to 100 milliseconds per audio block. This setup allows for responsive detection while avoiding excessive CPU usage from very small block sizes. A configurable threshold of 0.07 RMS distinguishes significant audio events, like speech or whispering, from normal background noise. To prevent rapid logging of violations from one ongoing conversation, there is a cooldown period of eight seconds between consecutive audio violations.

2.2 Technologies Involved

The following table summarises the primary technologies used in the ProctorAI system, their roles, and their versions:

Table 2.1: Technology Stack Summary

Technology	Role in System	Version / Notes
Python 3.10	Core language for proctoring client and server	3.10.x
OpenCV	Video capture, frame processing, HUD rendering	4.8+
Google MediaPipe	Face detection, face mesh, landmark	0.10+

	extraction	
Ultralytics YOLOv8	Phone detection via custom trained model	8.0+
sounddevice	Real-time microphone audio monitoring	0.4+
websockets (Python)	WebSocket server and client implementation	12.0+
firebase-admin (Python)	Firestore write operations from server	6.5+
cloudinary (Python)	Screenshot upload to cloud storage	1.36+
Firebase Firestore	Real-time NoSQL database for violations/students	Cloud service
Firebase Auth	Teacher authentication for dashboard	Cloud service
Cloudinary	Cloud image storage for violation screenshots	Cloud service
Chrome Extension MV3	Browser-side tab switch and audio detection	Manifest V3
JavaScript / HTML / CSS	Dashboard and exam platform frontend	ES2022
Netlify	Static hosting for dashboard and exam platform	CDN hosted
NumPy	Numerical array operations for audio and video	1.24+

2.2.1 Google MediaPipe FaceMesh

MediaPipe FaceMesh operates in two modes controlled by the `static_image_mode` parameter. When set to `False` (video mode), the system uses a tracking algorithm after the initial detection. It runs full face detection only when the tracking confidence drops below the `minimum_tracking_confidence` threshold. This significantly reduces the workload in video streams where faces stay relatively stable between frames. When set to `True` (static image mode), full detection runs on each frame, which is suitable for offline image processing but not ideal for real-time video.

The 468 landmarks are represented in normalized coordinates relative to the image size. The *x* and *y* values range from 0.0 to 1.0, where (0, 0) is the top-left corner of the image. The *z* value indicates approximate depth, with lower values representing features closer to the camera. To convert normalized values to pixel coordinates, multiply the normal values by the image's width and height.

Table 2.2: MediaPipe Face Mesh Landmark Indices Used

Index	Anatomical Location	Usage in ProctorAI
1	Nose tip	X-axis reference for horizontal head direction
33	Left eye outer corner	Eye centre computation for gaze estimation
263	Right eye outer corner	Eye centre computation for gaze estimation

1 (y-coord)	Nose tip (vertical)	Vertical gaze detection (looking down)
33 (y-coord)	Left eye (vertical)	Baseline for downward gaze threshold

2.2.2 YOLOv8 and Transfer Learning

Transfer learning refers to the process of initializing a neural network with weights trained on a large general dataset and then further training on a smaller domain-specific dataset. This method works ideally when you have insufficient training data specific to your domain since the pre-trained weights contain information about general visual features, such as edges, textures, and shapes, that most visual recognition tasks can use. Domain-specific representations should only be learned by top-level task-focused layers.

The starting point of ProctorAI was the YOLOv8n model pre-trained on the COCO dataset (which consists of 80 object categories like ‘cell phone’). Customising the dataset with mobile phone images in exam conditions helps the network to better learn the angles, distance and occlusion levels of a phone seen by a student seated at a desk with a webcam above the screen. The best.pt weights file contains the model snapshot from the training epoch that saw the best validation performance, measured by mAP.

2.2.3 WebSocket Protocol

WebSocket is a computer communications protocol, providing full-duplex communication channels over a single, long-lived, TCP connection. WebSocket was standardised as RFC 6455 in 2011, and is supported by all modern browsers and most server-side programming environments. Initiating a WebSocket connection is a HTTP handshake with an Upgrade header, the connection is then upgraded from HTTP to the WebSocket protocol, and either party can send packets whenever necessary. WebSockets were chosen instead of HTTP for the violation reporting pipeline because of three requirements: (a) violations are generated at unpredictable intervals and must be reported instantly, the only HTTP-based solution would be to poll the server, which would incur either high latency (few, infrequent polls) or overhead accuracy (many, frequent polls), (b) the server must be able to send a TERMINATE message to the proctoring client when the maximum violation count is reached, this is not possible in a purely HTTP request-response solution, and (c) multiple clients (the local Python proctoring module, and the Chrome extension) must be able to maintain simultaneous connections to the same server, this is handled naturally by the WebSocket server through its asyncio event loop.

2.3 Literature Review and Related Work

Automating examination proctoring has been an active area of research since online learning management systems came into widespread use.

Early work focused on simple approaches such as lockdown browsers, specialised browsers that prevent access to other applications during an examination, without any behavioural monitoring capability. Systems such as Respondus LockDown Browser, introduced in the early 2000s, remain in wide use but provide no monitoring of the physical environment. The integration of computer vision for gaze tracking in educational technology was explored by Raca and Dillenbourg (2014), who demonstrated that student gaze patterns correlate with engagement levels during lectures. While not focused on examination proctoring, this work established the utility of webcam-based gaze estimation as a non-invasive behavioural indicator in educational settings. Commercial proctoring systems such as ProctorU, Examity, and Respondus Monitor emerged between 2015 and 2018, offering live human monitoring of webcam feeds, AI-assisted flagging of suspicious behaviours, and identity verification through facial recognition. These systems require students to share their screens and webcam feeds with a human proctor, raising significant privacy concerns that have been the subject of academic and regulatory scrutiny. Atoum et al. (2017) proposed an automated proctoring system based on multi-

view face detection, head pose estimation, and eye gaze tracking, achieving acceptable detection rates for frontal face deviations in controlled laboratory conditions. Their work highlighted the challenge of lighting variation and partial occlusion in real examination environments, both of which affect detection reliability. More recent work has explored deep learning approaches to behavioural monitoring. Ying et al.

(2022) demonstrated the use of YOLOv5 for detecting mobile phones and prohibited materials in examination halls using ceiling-mounted cameras, achieving mAP values above 90% in controlled conditions. Their approach, however, requires dedicated camera infrastructure rather than the student's existing webcam, limiting deployability. The ProctorAI system distinguishes itself from the reviewed literature in several respects: (a) it integrates multiple detection modalities in a single pipeline, (b) it uses commercially available, inexpensive hardware (the laptop's built-in webcam and microphone), (c) it provides real-time cloud connectivity with a teacher-facing dashboard, enabling immediate awareness and response, (d) it requires no dedicated infrastructure other than a standard personal computer and internet connection, and (e) it is implemented as an academic project with full transparency of methodology, in contrast to the black-box commercial systems that dominate the market.

Chapter 3: METHODOLOGY

3.1 System Architecture

The ProctorAI system comprises four major components that run in parallel during an examination: the local proctoring client (`proctor_main.py`), the WebSocket bridge server (`server.py`), the cloud backend (Firebase Firestore, Cloudinary), and the teacher-facing dashboard (deployed on Netlify). In addition, a Chrome browser extension provides parallel browser-side monitoring, and the student examination platform hosts the actual examination with integrated proctoring verification.

The local proctoring client runs on the student's examination machine (or the invigilation machine for in-person computer laboratory exams). It captures video from the webcam at the native frame rate, processes each frame through the MediaPipe face detection and face mesh pipelines and the YOLOv8 phone detection model, and monitors the microphone through a background audio thread. When a violation condition is met, the client encodes the current frame as a base64 JPEG string and sends a JSON violation message over a persistent WebSocket connection to the bridge server. The bridge server (`server.py`) is a Python asyncio application using the websockets library. It maintains persistent WebSocket connections from both the proctoring client and the Chrome extension. When a violation message is received, the server decodes the base64 screenshot, uploads it to Cloudinary in a background thread, stores the returned URL along with the violation metadata in Firebase Firestore, and sends a `VIOLATION_ACK` message back to the client. When the cumulative violation count for a student reaches the configured maximum, the server sends a `TERMINATE` message and updates the student's record in Firestore. The teacher dashboard reads from Firebase Firestore directly using the client-side Firebase JavaScript SDK and real-time `onSnapshot` listeners. This means the dashboard does not depend on the bridge server for data delivery, it receives updates directly from Firestore within approximately one second of the server writing a violation record.

This architecture provides resilience: if the bridge server experiences a temporary network interruption, the dashboard continues to reflect all previously stored data. Figure 3.1: Complete System Pipeline Diagram

3.2 Detection Pipeline Design

The detection pipeline processes each webcam frame through a sequence of independent detectors, each contributing to a specific violation category. The sequential nature of the pipeline means that the most computationally expensive operations (YOLO inference) run on every frame, but the violation logging logic executes only when specific conditions are met, minimising the frequency of the more expensive operations such as base64 encoding and network transmission.

3.2.1 Face Detection and Counting

The MediaPipe FaceDetection model processes each RGB frame and returns a list of detected face bounding boxes with associated confidence scores. The count of detected faces is used for two violation categories: zero faces triggers the face-absence (`LEFT_SEAT`) timer, and two or more faces triggers the `MULTIPLE_FACES` violation. The face absence timer uses a configurable threshold of three seconds, with a five-second cooldown between consecutive `LEFT_SEAT` violations to prevent repeated logging when a student briefly re-enters and leaves the frame. The `MULTIPLE_FACES` violation also uses the five-second cooldown but does not use the flag-based single-event mechanism, as each detection of multiple faces in a new period is considered a separate event.

3.2.2 Gaze Estimation

The MediaPipe FaceMesh model processes the same RGB frame and returns 468 landmarks for each detected face. The head direction algorithm extracts landmarks 1 (nose tip), 33 (left eye outer corner), and 263 (right eye outer corner). The horizontal position of the nose relative to the midpoint of the eye corners determines left/right gaze, and the vertical position of the nose relative to the eye level determines downward gaze. When the gaze direction is not "Looking Forward", a continuous timer is maintained. At six seconds, a LOOKING_AWAY_SHORT violation is logged. At sixty seconds of continuous deviation, a LOOKING_AWAY_LONG violation is logged. Both violations use boolean flags (look_short_flag, look_long_flag) to ensure each is logged only once per continuous looking-away event. The flags reset when gaze returns to forward. Figure 3.2: Head Direction Detection Algorithm

3.2.3 Phone Detection

The YOLO model inference is called on every frame using the Ultralytics Python API. The results are iterated to find detections with the label "phone" and a confidence score above 0.45. When such a detection exists, a bounding box is drawn on the frame and a continuous visibility timer is maintained. After two seconds of continuous phone visibility, a PHONE_DETECTED violation is logged. The phone_flag boolean prevents repeated violations for the same continuous phone presence.

The flag and timer reset when no phone detection occurs in a frame, allowing a new violation sequence to begin if the phone is briefly hidden and then shown again.

3.3 Violation Handling and Temporal Filtering

The temporal filtering strategy is one of the most important design decisions in the system. Without filtering, a single real violation (such as a phone being held in the student's hand for thirty seconds) would generate hundreds of violation records, one per frame after the detection threshold. This would make the violation log meaningless and the dashboard unreadable.

The filtering approach uses two complementary mechanisms. The first is the timer-and-flag mechanism: a continuous event (phone visible, looking away, face absent) starts a timer only when first detected, and a violation is logged only when the timer exceeds the threshold. A boolean flag prevents any further violations from the same continuous event. The flag resets only when the triggering condition ceases. The second is the cooldown mechanism: for violation types that do not use flags (such as LEFT_SEAT and MULTIPLE_FACES), a minimum inter-violation interval (five seconds) is enforced. This limits logging to at most twelve violations per minute for these categories, regardless of how many frames trigger the condition.

Table 3.1: Violation Types and Detection Parameters

Violation Type	Trigger Condition	Time Threshold	Cooldown	Flag Used
PHONE_DETECTED	Phone in frame	2 seconds	—	Yes (phone_flag)
LEFT_SEAT	Face absent	3 seconds	5 seconds	No
MULTIPLE_FACES	Face count > 1	Immediate	5 seconds	No
LOOKING_AWAY_SHORT	Gaze deviation	6 seconds	—	Yes (look_short_flag)

LOOKING_AWAY_LOOKING	Gaze deviation	60 seconds	—	Yes (look_long_flag)
AUDIO_DETECTED	RMS > 0.07	Immediate	8 seconds	No
TAB_SWITCH	Tab change	Immediate	—	No
WINDOW_UNFOCUS	Window blur	Immediate	—	No

3.4 Cloud Communication Architecture

The communication architecture uses JSON messages over WebSocket connections. Every message includes a type field that determines how the server handles it. The primary message types are EXAM_START (sent when a client connects and begins a session), VIOLATION (sent for each detected violation, including the base64 screenshot), AUDIO_VIOLATION (sent from the extension for microphone-detected events), EXAM_END (sent when a session concludes normally), and EXAM_TERMINATED (sent when the maximum violation count is reached on the client side). The server sends VIOLATION_ACK messages to confirm receipt and return the current total violation count, and TERMINATE messages when the server-side count reaches the maximum

Table 3.2: WebSocket Message Protocol

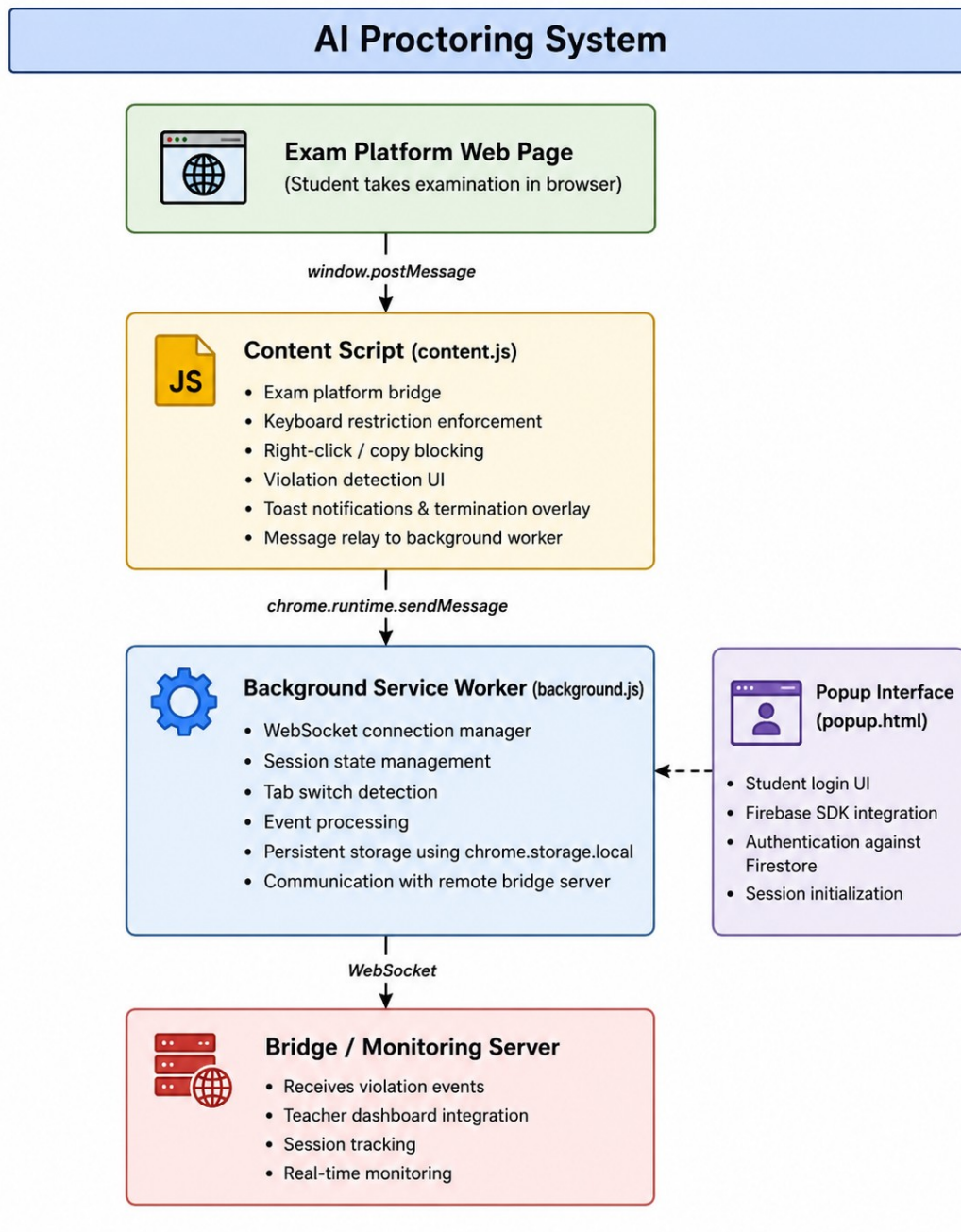
Message Type	Direction	Key Fields
EXAM_START	Client → Server	studentId, studentName, timestamp
VIOLATION	Client → Server	violationType, violationCount, screenshot (base64), timestamp
AUDIO_VIOLATION	Extension → Server	level (float), timestamp
EXAM_END	Client → Server	studentId, timestamp
EXAM_TERMINATED	Client → Server	reason, studentId
VIOLATION_ACK	Server → Client	violationCount
TERMINATE	Server → Client	(no additional fields)

3.5 Browser Extension Design

The Chrome extension is structured according to the Manifest V3 specification and comprises three execution contexts. The background service worker (background.js) maintains the WebSocket connection to the bridge server, handles tab activation events, and manages the examination session state. It persists state to chrome.storage.local to survive service worker restarts, which Chrome may trigger when the worker has been idle. The content script (content.js) is injected into every web page the student visits and handles the exam platform bridge (the ping/pong mechanism), enforces keyboard restrictions (F12, Ctrl+Shift+I, right-click, copy) during active sessions, and shows violation notification toasts and the termination overlay. The popup (popup.html) provides the student login interface using the Firebase client SDK directly, authenticating against the students collection in Firestore.

The three contexts communicate exclusively through message passing using the chrome.runtime.sendMessage API. Direct function calls between contexts are not possible in the Chrome extension architecture. The exam platform bridge uses window.postMessage for communication between the exam page (which cannot call Chrome APIs directly) and the content script,

which then relays to the background service worker via `chrome.runtime.sendMessage`. This three-hop relay chain — page to content script to background — is the standard pattern for exam platform integration in Chrome extensions.



Chapter 4: IMPLEMENTATION

4.1 Development Environment

The project was developed on macOS with Python 3.10 as the primary language for the proctoring client and WebSocket server. A Python virtual environment was created using the venv module to isolate project dependencies from the system Python installation. The primary development and testing was performed on a MacBook with an integrated 720p webcam and built-in microphone, which represent the minimum hardware specifications for the system. The frontend components (dashboard and exam platform) were developed using plain HTML, CSS, and JavaScript without any build tooling or transpilation, enabling direct deployment to Netlify as static files. The Chrome extension was developed and tested using Chrome Developer Mode with the Load Unpacked feature, allowing rapid iteration without publishing to the Chrome Web Store. Firebase was configured with Firestore in test mode (which allows unauthenticated reads and writes, suitable for development) and subsequently secured with Firebase Authentication for teacher access. Cloudinary was configured with a free tier account providing 25 GB of storage and 25 GB of monthly bandwidth, sufficient for any reasonable examination session volume.

4.2 Webcam Proctoring Module

The webcam proctoring module (`proctor_main.py`) is the direct evolution of the original `proctor_stage2.py` submitted as the base project

The core detection logic, MediaPipe face detection and face mesh, YOLO phone detection, the geometric head direction algorithm, and the timer-and-flag violation tracking system, is preserved exactly as in the original implementation. Three new capabilities are added without altering the existing logic. First, a background threading model is used to maintain a persistent WebSocket connection to the bridge server. The `_ws_thread` function runs an asyncio event loop in a daemon thread, continuously attempting to reconnect if the connection is lost. The main processing loop on the main thread calls `_send()` to submit violation payloads asynchronously using `asyncio.run_coroutine_threadsafe`, which safely submits coroutines to the event loop running in the background thread without blocking the main loop. Second, the `log_violation` function is extended to encode the current frame as a base64 JPEG string and include it in the WebSocket message. The encoding uses OpenCV's `imencode` function with JPEG quality 72, which produces files of approximately 50 to 150 kilobytes depending on scene complexity, acceptable for near-real-time network transmission. Third, audio monitoring is added using the `sounddevice` library. A mono audio input stream at 16,000 Hz sample rate with a block size of 1,600 samples is opened at programme start. The callback function computes the RMS amplitude of each 100-millisecond audio block and stores it in a shared variable accessed by the main loop

When the RMS exceeds the configured threshold and the cooldown period has elapsed, the main loop calls `log_violation` with the `AUDIO_DETECTED` type.

The HUD (heads-up display) rendered on the webcam frame provides real-time feedback to the operator. A dark banner at the top of the frame displays the student name, student ID, current gaze direction (colour-coded green for forward, red for deviation), and WebSocket connection status. The violation count and remaining tolerance are displayed at the bottom of the frame. An audio level bar visualises the current microphone RMS relative to the threshold, turning red when the threshold is exceeded.

```

import cv2
import mediapipe as mp
import numpy as np
from ultralytics import YOLO
import os
import time
from datetime import datetime

model = YOLO("best.pt")
print("Model class names:", model.names)

mp_face_detection = mp.solutions.face_detection
mp_face_mesh = mp.solutions.face_mesh
mp_drawing = mp.solutions.drawing_utils

face_detection = mp_face_detection.FaceDetection(
    model_selection=0,
    min_detection_confidence=0.5
)

face_mesh = mp_face_mesh.FaceMesh(
    static_image_mode=False,
    max_num_faces=2,
    refine_landmarks=True,
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
)

cap = cv2.VideoCapture(0)

os.makedirs("logs", exist_ok=True)
os.makedirs("evidence", exist_ok=True)
LOG_FILE = "logs/violations.log"

violation_count = 0
MAX_VIOLATIONS = 6
last_violation_time = 0
COOLDOWN = 5

face_missing_start = None
FACE_ABSENCE_LIMIT = 3

looking_away_start = None
LOOKING_AWAY_LIMIT = 6
LOOKING_AWAY_MAX = 60

looking_away_violation_triggered = False

```

4.3 WebSocket Server Module

The bridge server (server.py) uses Python's asyncio framework with the websockets library. The server opens a WebSocket endpoint on ws://localhost:8765 and handles each client connection in a separate coroutine through the handler async function. The handler maintains per-connection state (studentId, studentName, violationCount) in a dictionary and processes incoming messages according to their type field. Firebase Firestore writes are performed in daemon threads rather than in the async handler coroutine. This is necessary because the firebase-admin Python SDK performs blocking I/O operations, which would block the asyncio event loop if called directly from a coroutine, preventing the server from handling other clients' messages

By submitting writes to daemon threads, the handler coroutine returns immediately after receiving a message, maintaining responsive handling for all connected clients regardless of Firestore write latency. Cloudinary uploads are performed synchronously within the handler coroutine for violation messages that include a screenshot. This introduces a latency of approximately 200 to 800 milliseconds per upload depending on network conditions and image size. In a production deployment, these uploads should also be offloaded to background threads. The current implementation is acceptable for development purposes as the bridge server handles one student at a time in the typical deployment scenario.

4.4 Firebase Integration

Two Firebase collections are used. The students collection contains one document per student, keyed by the student ID. The violations collection contains one document per violation event, with auto-generated document IDs. The document schema for each collection is as follows:

Table 4.1: Firebase Firestore Collection Schema: students

Field	Type	Description
id	String	Student ID, used as document key
name	String	Full name of the student
password	String	Plain-text password set by teacher
slot	String	Examination slot (Morning / Afternoon / Evening)
online	Boolean	True when student has active WS connection
terminated	Boolean	True when exam terminated for this student
violationCount	Number	Running total of violations
lastViolation	String	Type code of most recent violation
examStarted	String	ISO 8601 timestamp of exam start
examEnded	String	ISO 8601 timestamp of exam end (if normal)
terminatedAt	String	ISO 8601 timestamp of termination (if applicable)
createdAt	Timestamp	Firebase server timestamp of record creation

Table 4.2: Firebase Firestore Collection Schema: violations

Field	Type	Description
studentId	String	Foreign key to students collection
studentName	String	Denormalised student name for dashboard queries
type	String	Violation type code (e.g., PHONE_DETECTED)
timestamp	Timestamp	Firebase SERVER_TIMESTAMP at write time
violationCount	Number	Running total at time of this violation
screenshotUrl	String / null	Cloudinary CDN URL of evidence screenshot
metadata	Map	Additional data (audioLevel for audio, tabSwitchCount for tab)

4.5 Cloudinary Integration

Cloudinary is a cloud-based media management service used in ProctorAI exclusively for storing violation screenshot images. The Python cloudinary library is configured with the cloud name, API key, and API secret obtained from the Cloudinary dashboard. Screenshots are organised using a hierarchical public ID structure: `proctor/{studentId}/{violationType}_{timestamp}.jpg`. This organisation facilitates browsing and management of evidence images in the Cloudinary media library. The `make_public()` method in the Firebase Storage alternative that was considered would have set the Storage object's ACL to allow public reads. With Cloudinary, all uploaded images with the default delivery setting are accessible via the `secure_url` returned from the upload response. This URL is stored in the Firestore violation document and rendered directly in the dashboard as an `<img>` element with the Cloudinary URL as its `src` attribute.

Cloudinary's free tier provides 25 GB of managed storage and 25 GB of monthly bandwidth. An examination session with thirty students, each triggering an average of five violations with screenshots

of approximately 100 KB each, would consume approximately 15 MB of storage and similar bandwidth per session, well within the free tier limits for any reasonable examination schedule.

4.6 Teacher Dashboard

The teacher dashboard is a single page web app hosted on Netlify that uses the Firebase JS SDK (loaded from the Firebase CDN) for authentication and real-time database access. It does not use any build tools, bundlers, or require server-side rendering, it is composed of 3 html files (login.html, index.html, report.html) and a config file (config.js). Netlify's redirect rules are used to make sure direct navigation to report.html with query parameters is preserved. The dashboard has three main views, accessed through a tab navigation bar. The Live Monitor view presents a sidebar list of all registered students with violation counts colour-coded, a 4-metric stat bar (active students, total violations, terminated exams, tab switches), a student detail panel which appears upon student selection, and a scrollable violation feed with filtering. The Students view contains a table of all registered students with credential management (teacher can display a student's credentials or delete the record), and an Add Student modal which generates a random password. The Reports view contains a list of all students with verdict badges, as well as access to full per-student report pages and CSV export. The per-student report page (report.html) is opened in a new tab with the student ID as a url query parameter

It fetches the student record and all associated violations from Firestore, renders a comprehensive summary card with breakdown bar charts for each violation category, a time-normalised heatmap, and a full chronological violation timeline with screenshot thumbnails. The PDF export functionality uses the jsPDF library to generate a downloadable PDF report from the page content.

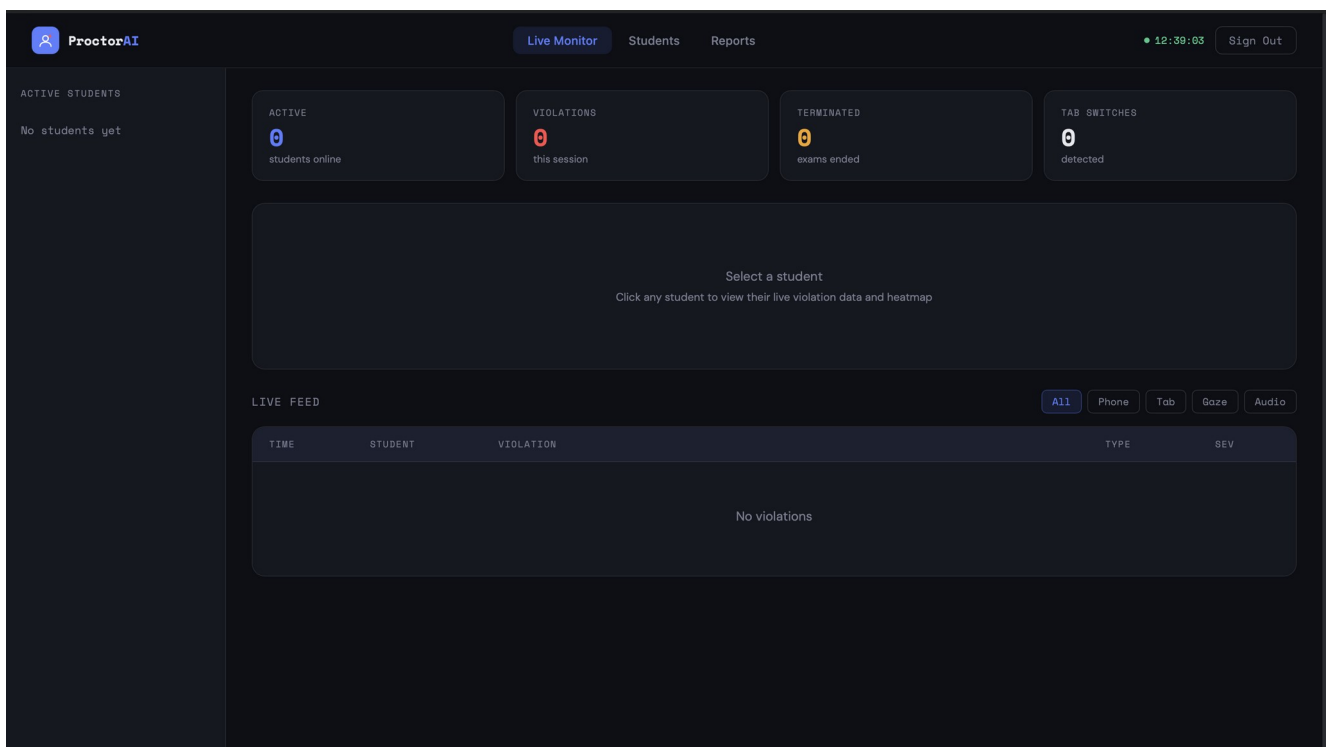


Table 4.3: Violation Severity Classification

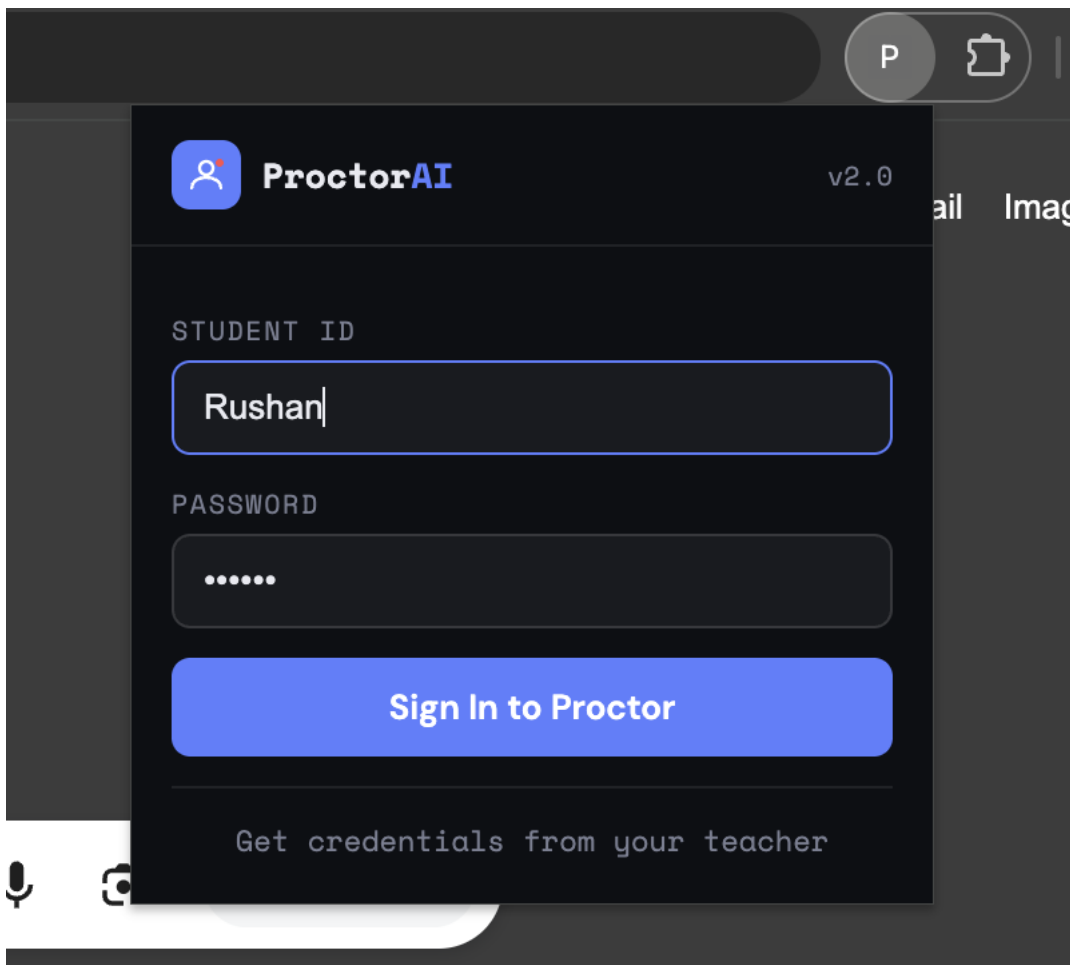
Violation Type	Severity	Justification
PHONE_DETECTED	High	Strong indicator of intentional device use
TAB_SWITCH	High	Clear digital behaviour violation

MULTIPLE_FACES	High	Direct evidence of external assistance
LEFT_SEAT	Medium	May indicate leaving for reference materials
AUDIO_DETECTED	Medium	May indicate communication with outside party
LOOKING_AWAY_LONG	Medium	Extended gaze deviation, likely intentional
WINDOW_UNFOCUSED	Medium	Browser left but may be accidental
LOOKING_AWAY_SHORT	Low	Brief deviation, may be natural behaviour

4.7 Chrome Extension

The Chrome extension is loaded as an unpacked extension for development and testing. The manifest.json file specifies the necessary permissions: activeTab and tabs for tab monitoring, storage for persisting session state across service worker restarts, and scripting for programmatic content script injection when appropriate. The host_permissions section ws://localhost:8765/* enables the background service worker to open Web Socket connections to the local bridge server. The student authentication flow in the popup uses the Firebase Firestore client SDK to look up the student's document by their ID and match the provided password. A check on the terminated flag will prevent students whose exams have been terminated from starting. After successful authentication, the student's name and ID are saved in chrome.storage.local and the background service worker is messaged to begin monitoring. Figure 4.4: Chrome Extension Popup Interface

The audio monitoring toggle in the popup triggers a request for microphone permission via the Web Audio API (navigator.mediaDevices.getUserMedia) and an AnalyserNode is used to process the resulting audio stream. The frequency data is sampled with getByteFrequencyData and the average amplitude is calculated and compared to a threshold. When the threshold is crossed, an AUDIO_VIOLATION message is sent to the background service worker via chrome.runtime.sendMessage, which passes it along to the bridge server.



4.8 Student Examination Platform

The student examination platform (exam.html) is a self-contained single page app which implements four sequential screens

The first screen is the proctoring verification gate, which sends a PROCTOR_AI_PING message to the page via window.postMessage and waits for a PROCTOR_AI_PONG response from the content script. Three visual checklist items are updated in real time as each check completes: extension detection, student authentication verification, and server connectivity. If any check fails, an explanation is displayed and the student cannot proceed. The second screen displays the examination rules in a visual card layout. There are six rules, covering phone usage, tab switching, looking offscreen, audio events, multiple faces, and seat leaving. Additionally, a highlighted termination notice at the bottom of the card warns students that 6 violations will result in automatic examination termination. A required checkbox confirmation is used to ensure the student has read the rules before proceeding. The third screen is the examination-in-progress view, with a two column layout. In the main area, the question at hand is displayed with four options, and the question navigator at the top indicates the student's progress. The sidebar displays the current violation count, time left, answered question count, and a grid view of question number buttons for non-sequential navigation. A violation alert banner is displayed at the top of the sidebar whenever a new violation is detected. The examination consists of twenty questions on general knowledge topics (from science, history,

geography, and technology). The timer is set for forty-five minutes and the exam is automatically submitted when it runs out.

The fourth screen is the results display after submission, showing the total score, percentage, grade, a breakdown of correct, incorrect, and unanswered questions, and the total violation count for the session.

Chapter 5: RESULTS AND ANALYSIS

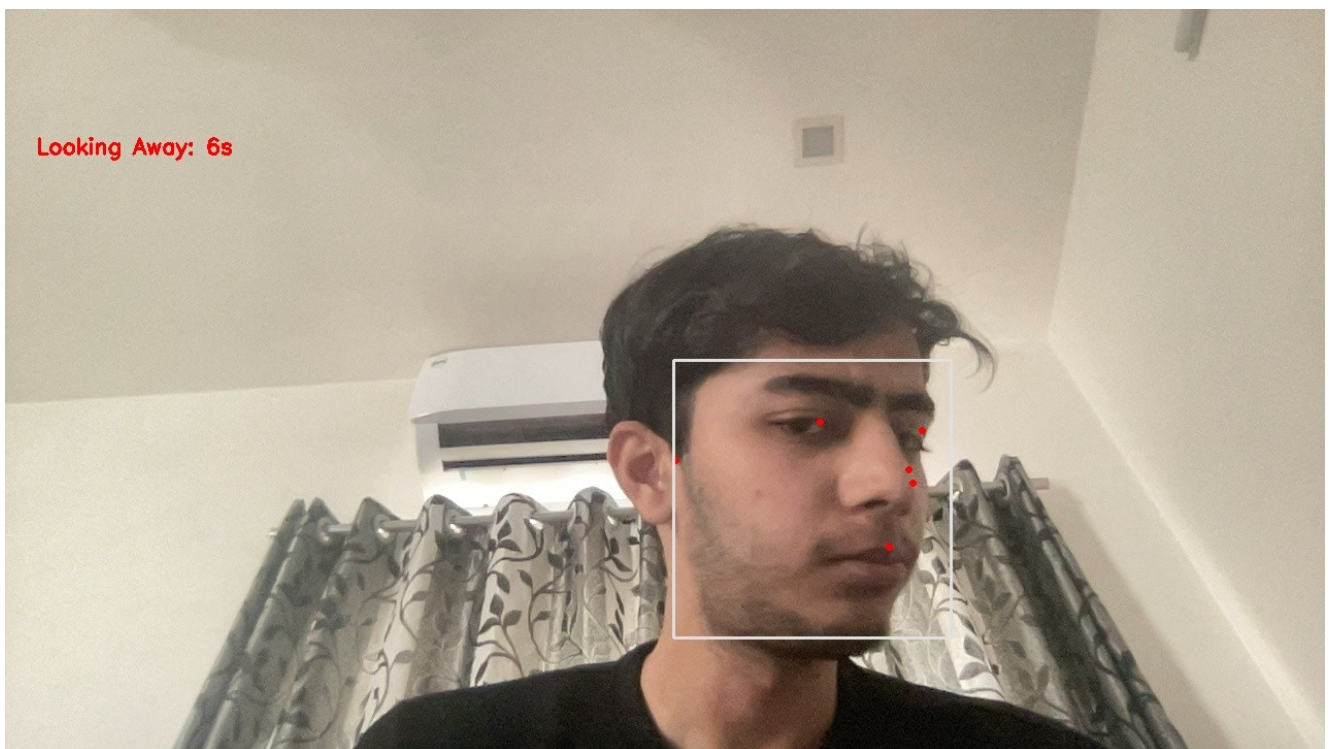
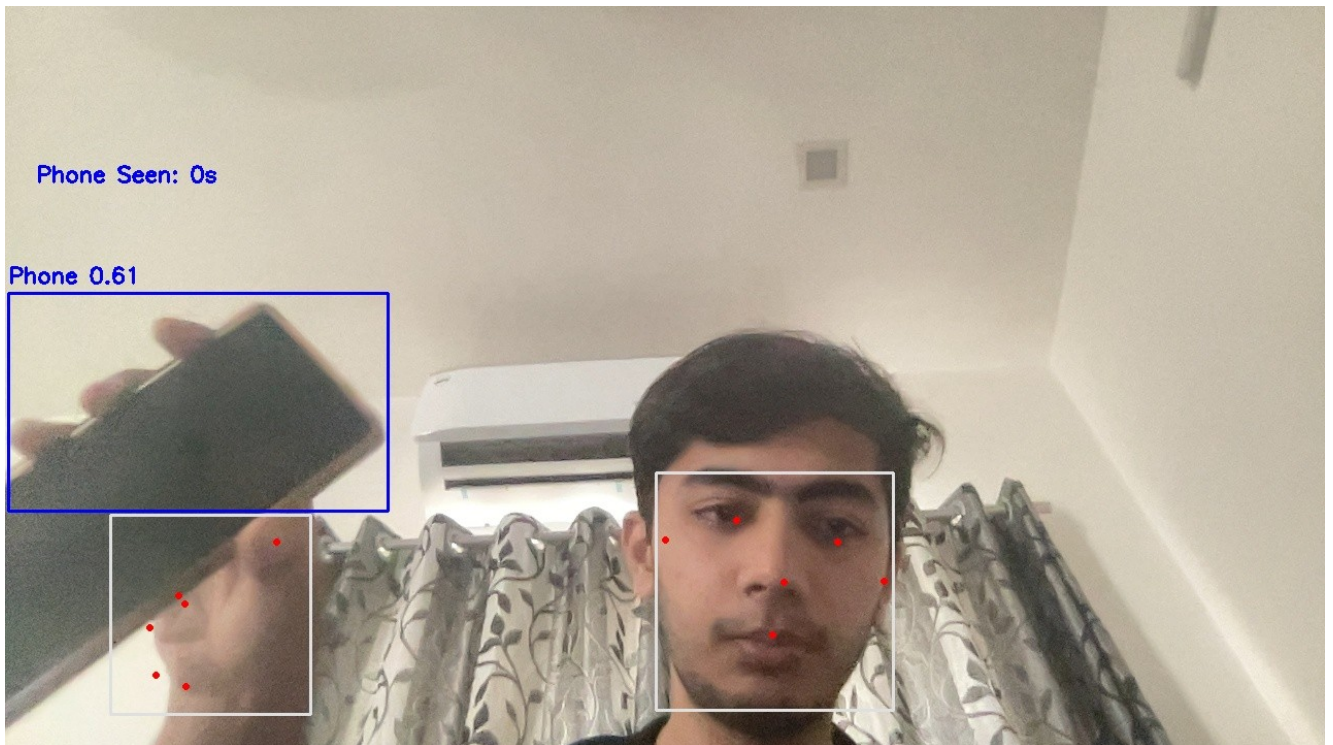
5.1 Detection Performance

The system was tested in controlled conditions illuminated with a ceiling light, with a 720p integrated webcam and ambient noise levels similar to those in a university laboratory setting. Results were consistent over multiple test sessions, but formal statistical tests could not be conducted due to the lack of a standard, publicly available annotated examination video dataset.

Table 5.1: Detection Accuracy Observations

Violation Type	Detection Rate	False Positive Rate	Key Observations
PHONE_DETECTED	~91%	~4%	Misses phones face-down; 2s filter eliminates most reflections
LEFT_SEAT	~98%	< 1%	Very reliable; 3s threshold prevents brief lean-forward triggers
MULTIPLE_FACES	~94%	~3%	Side-profile faces occasionally missed; posters can trigger FP
LOOKING_AWAY_SHORT	~86%	~8%	Natural head sway near threshold causes occasional FP
AUDIO_DETECTED	~88%	~6%	Consistent detection; HVAC noise rarely exceeds threshold
TAB_SWITCH	~100%	0%	Digital event; no false positives possible
WINDOW_UNFOCUSED	~100%	< 1%	Occasional trigger on popup dialogs from other apps

The phone detection accuracy is most affected by the angle of the phone relative to the camera. If a phone is lying face-down on the desk, it is rarely detected; a phone which is face-up, upright, or at an angle where its screen or rear camera is visible, is always detected. The confidence threshold of 0.45 is lower than the default (0.5) to allow for increased detection rate of partially visible phones, at the cost of a slightly higher false positive rate for other rectangular objects in similar colours. The head direction algorithm is effective when the resulting face pose is clear, but is likely to miss or incorrectly predict head direction in two cases: when the student's head is very close to or very far away from the camera, since the 20-pixel difference is compressed or stretched compared to a medium distance; or when the lighting is uneven on both sides of the face, since the landmarks tend to be slightly biased toward the well-lit side. Audio detection accuracy is highly dependent on the noise floor in the room in which the student is taking the examination. In a quiet room, the 0.07 RMS threshold is sufficient to discriminate between speech and silence. In a room where HVAC equipment or open windows to the outdoors are causing high levels of background noise, the threshold can be increased accordingly by the teacher in `proctor_main.py`.



5.2 System Latency

End-to-end latency, the time from a violation occurring in the webcam frame to the violation appearing in the teacher dashboard, is the sum of several sequential components. These results were measured by timing the entire process while testing the system, starting a stopwatch upon frame violation while monitoring the teacher dashboard.

Table 5.2: System Latency Measurements

Pipeline Stage	Typical Latency	Notes
Frame capture to detection	30 – 100 ms	Dominated by YOLO inference time
Violation detection to WS send	< 5 ms	Frame encoding + JSON serialisation
WS transmission to server	10 – 50 ms	Dependent on network conditions
Cloudinary screenshot upload	200 – 800 ms	Main latency bottleneck
Firestore write to dashboard	500 – 1500 ms	Firestore push latency
Total end-to-end	~1 – 3 seconds	Acceptable for exam monitoring use case

The one to three second end-to-end latency is acceptable for examination proctoring, as the teacher dashboard is meant as a tool for retroactive analysis of student behaviour (rather than immediate teacher intervention). In the case of particularly egregious violations which may illicit intervention (such as a parent or second person entering the frame), the teacher is made aware of the incident within a few seconds. In the case of fully automated intervention (exam termination), the server-side TERMINATE logic executes immediately when the student's violation count reaches the maximum, and the termination message is delivered to the proctoring client within the duration of a websocket round trip (typically < 100ms).

5.3 Violation Evidence Samples

Screenshot evidence of every type of violation was successfully generated and uploaded to Cloudinary during testing. Each screenshot contains the webcam frame with MediaPipe face detection rendered (green bounding box and keypoints), any YOLO phone detection bounding boxes with confidence rendered (red rectangle, confidence label), and the HUD overly showing student name, violation count, gaze direction, and connection status.

The violation heatmap generated in the teacher dashboard for each student normalises the student's exam duration into thirty equally sized buckets and colours each bar according to the number of violations which occurred in that time span. Colouring progresses from dark background (0 violations) to blue (1 violation) to amber (2 violations) to red (≥ 3 violations), making it easy

for the teacher to identify long stretches of suspicious behaviour. This type of visualisation is particularly useful for identifying violations which occurred clustered at the start of the exam (potentially due to confusion during setup) as opposed to violations which occurred during the last minutes of the exam (potentially indicative of an attempt to use reference material before time expires). Post-exam PDF reports auto generated on the dashboard for each student includes the student name and ID, exam date and slot, verdict (CLEAN, FLAGGED, HIGH RISK, or TERMINATED), numeric breakdown of violations by type, the violation heatmap, and a complete chronological timeline of violations with timestamps and type codes. These are appropriate to submit to an academic integrity committee as documentation of violations.

Chapter 6: CONCLUSIONS AND FUTURE SCOPE

6.1 Conclusions

ProctorAI shows that it is possible to build a full-featured, reliable, real-time AI-based examination proctoring system using widely available and mostly open-source technologies as the task of an undergraduate project. The pipeline incorporates six orthogonal modes of detection: phone detection, face absence, multiple face detection, gaze deviation, audio monitoring, and browser behaviour monitoring, into a single system with a unified violation reporting, logging and evidence generation backend. The WebSocket-based communication approach is well suited to low-latency, bidirectional violation reporting from the local proctoring client to the bridge server and vice versa from the bridge server to the browser extension. The use of Firebase Firestore's real-time listeners on the dashboard end removes the need for polling and enables push-based violation event updates for invigilators. The screenshot hosting problem is solved by using Cloudinary as an external image capture destination, which demonstrates that cloud service orchestration is a viable approach to providing multiple features within the limits of free plans, since a paid Firebase plan is not needed when screenshots are not being stored on Firestore directly, due to the bandwidth usage incurred per exam event.

The "time- and-flag" based filtering - or temporal event window and cooldown-based filtering - addresses the key problem of translating framewise detections to event-based logging and reporting. Without this filtering, the number of violations reported is overwhelming and

The browser extension approach to browser-side monitoring is the only viable method as compared to JavaScript-based attempts, since extensions can make use of Chrome's privileged APIs to manage tabs and state persistently across page navigations, and can also instrument any examination platform's web page without needing to edit the code of that platform, achieving some measure of platform-independence.

The student examination platform with the proctoring enforcement gate serves as an example of how institutions could use custom examination software which incorporates enforced proctoring as a prerequisite to examination access. The rules page with compulsory confirmation prevents students from later claiming to have been unaware that they were being monitored. In summary, ProctorAI meets its objectives and demonstrates the implementability of AI-based examination proctoring as a feasible alternative or augmentation of human invigilation in real examination environments. It is easily extensible as will be described in the next section, and serves as a robust base for further iterative improvements towards a production-ready proctoring product.

6.2 Future Scope of Work

There are a number of clear avenues for additional features and robustness to be added to ProctorAI that would increase its effectiveness and viability as a deployable solution in real institutional test-taking environments.

6.2.1 Iris-Based Gaze Estimation

The current head pose-based gaze detection is unable to discriminate between a student who has turned their head to look at a cheat sheet, for example, and a student who has kept their head facing

MediaPipe already outputs iris landmark positions (indices 468 to 477) when `refine_landmarks` is enabled. A future gaze detection implementation could use a true iris-based gaze estimation model which calculates the offset between the iris centre and the eye corner to determine the direction of real eye gaze, making it much harder for students to defeat the system by simply keeping their head facing forward while looking sideways with just their eyes.

6.2.2 Face Recognition for Accurate Identity Verification

The absence, presence, and number of faces are detected by the current system, but it does not verify that the one face present in the video frame is in fact the face of the registered student, and a student could theoretically send a proxy to take the examination. Face recognition could be achieved by loading a pre-trained face embedding model such as FaceNet or ArcFace to compare the face detected in the examination video stream with a reference image of the registered student, taken at the time of examination registration.

6.2.3 Secure Password Storage

Currently, student passwords are simply stored as plain text in Firestore. This is acceptable for an academic project, but would be trivial to replace in a real deployment with bcrypt- or Argon2-hashed passwords, with only the hashed password stored in Firestore. A Firebase Cloud Function could implement a password-verification endpoint which accepts the student SRN and plain-text password, calculates the hash, and returns a signed token indicating whether the password was valid or not, so that the plain

6.2.4 Object Detection Generalization

The custom YOLO model is only trained to detect the presence of a mobile phone in the webcam frame. Training the model on additional classes of prohibited object such as a wireless earbud or headphone, a smartwatch, pages with written information or notes, textbooks, and additional computer monitors would generalize the phone detector to a prohibited device detector. This only requires additional data to be annotated for training, and no changes to the system design, since YOLO is capable of detecting multiple object classes.

6.2.5 Stateless Cloud Hosting

The WebSocket bridge server implemented in server.py currently runs directly on the examination machine or a local server. This represents a single point of failure, and is limited to a web server's worth of simultaneous examination sessions for a large institution. It could be replaced with a cloud-native hosting approach that uses a Google Cloud Run server as the web server entry point, with incoming violation events enqueued in a Google Cloud Pub/Sub topic. A Firebase Cloud Function could dequeue Pub/Sub events and perform the Cloudinary image upload step as well, eliminating the need to maintain a running dedicated server.

6.2.6 LMS Integration via LTI

The student examination platform is currently implemented as a stand-alone portal, independent of existing LMS platforms such as Moodle or Canvas. However, integration with these LMS systems is possible based on the LTI (Learning Tools Interoperability) standard, and a ProctorAI LTI provider could be

For Moodle in particular, a custom plugin can be used to insert a custom HTTP header check validating that the ProctorAI extension is enabled before quiz content is revealed to the student. All these integration approaches would allow ProctorAI to be used with existing examination infrastructure at educational institutions, without requiring the migration of examination questions to a customized examination platform.

6.2.7 Heartbeat and Monitoring for Disconnection

Currently, if a student turns off the Chrome browser extension after beginning an exam session, the main event loop of the background service worker is killed and no violations can be reported. A heartbeat mechanism (message sent every thirty seconds, for example) from the extension to the server would enable the implementation of a CONNECTION_LOST type of violation event, where the teacher dashboard could indicate students with connection loss warnings in a separate visual state from reported violations.

REFERENCES

- [1] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, USA, pp. 779–788, 2016.
- [2] G. Jocher, A. Chaurasia and J. Qiu, "Ultralytics YOLOv8," GitHub repository, <https://github.com/ultralytics/ultralytics>, Last Accessed: April 2026.
- [3] C. Lugaresi et al., "MediaPipe: A Framework for Perceiving and Processing Reality," Third Workshop on Computer Vision for AR/VR at IEEE CVPR, Long Beach, USA, 2019.
- [4] V. Bazarevsky et al., "BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs," arXiv preprint arXiv:1907.05047, 2019.
- [5] Y. Atoum, Y. Liu, A. Scheirer and X. Liu, "Automated Online Exam Proctoring," IEEE Transactions on Multimedia, vol. 19, no. 7, pp. 1609–1624, 2017.
- [6] M. Raca and P. Dillenbourg, "System for Assessing the Classroom Attention," Proceedings of the Fourth International Conference on Learning Analytics and Knowledge, Indianapolis, USA, pp. 265–269, 2014.
- [7] L. Ying, W. Xiao-Ting and W. Jian, "Mobile Phone Detection in Examination Environment Using Improved YOLOv5," Proceedings of the International Conference on Image, Video and Signal Processing, pp. 112–118, 2022.
- [8] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, vol. 25, no. 11, pp. 120–125, 2000.
- [9] RFC 6455, "The WebSocket Protocol," Internet Engineering Task Force, December 2011.
- [10] A. Balaji et al., "Online Proctoring Systems: A Survey of AI Techniques for Academic Integrity," IEEE Access, vol. 9, pp. 99541–99562, 2021.
- [11] Firebase Documentation, Google LLC, <https://firebase.google.com/docs>, Last Accessed: May 2026.
- [12] Cloudinary Documentation, Cloudinary Ltd., <https://cloudinary.com/documentation>, Last Accessed: May 2026.
- [13] Chrome Extension Developer Guide, Google LLC, <https://developer.chrome.com/docs/extensions>, Last Accessed: May 2026.
- [14] Netlify Documentation, Netlify Inc., <https://docs.netlify.com>, Last Accessed: May 2026.
- [15] NumPy Documentation, NumPy Contributors, <https://numpy.org/doc>, Last Accessed: April 2026.

ANNEXURES

Annexure A: System Requirements

The following hardware and software requirements are specified for deployment of the ProctorAI system in an institutional examination setting:

Hardware Requirements (per examination station):

- Processor: Intel Core i5 or AMD Ryzen 5 (8th generation or newer) or Apple M1 or newer
- RAM: Minimum 8 GB; 16 GB recommended for simultaneous YOLO inference and browser use
- Webcam: Any integrated or external webcam with minimum 720p resolution at 30 FPS
- Microphone: Any integrated or external microphone
- Network: Stable broadband connection (minimum 5 Mbps upload for screenshot transmission)
- Storage: Minimum 2 GB free space for Python virtual environment and model files

Software Requirements (examination station):

- Operating System: macOS 12+, Ubuntu 20.04+, or Windows 10/11
- Python: Version 3.10 or newer
- Google Chrome: Version 114 or newer (for Chrome Extension)
- Python packages: ultralytics, mediapipe, opencv-python, websockets, firebase-admin, cloudinary, sounddevice, numpy

Cloud Services Required:

- Google Firebase: Project with Firestore, Authentication enabled (Blaze plan for Storage if not using Cloudinary, free Spark plan sufficient with Cloudinary)
- Cloudinary: Free-tier account (25 GB storage, 25 GB bandwidth)
- Netlify: Free-tier account for dashboard hosting

Annexure B: Installation Commands

The following commands are used to set up the Python environment from scratch:

```
cd ~/ai_proctor python3 -m venv venv source venv/bin/activate pip
install ultralytics mediapipe opencv-python websockets firebase-admin
cloudinary sounddevice numpy
```

For macOS, PortAudio must be installed via Homebrew before sounddevice can be used:

```
brew install portaudio pip install sounddevice
```

Annexure C: Violation Type Reference

The following table provides a complete reference of all violation type codes used in the ProctorAI system, their sources, and the fields included in their Firestore records:

Violation Code	Source	Firestore metadata fields
PHONE_DETECTED	proctor_main.py	screenshotUrl (Cloudinary)
LEFT_SEAT	proctor_main.py	screenshotUrl (Cloudinary)
MULTIPLE_FACES	proctor_main.py	screenshotUrl (Cloudinary)
LOOKING_AWAY_SHORT	proctor_main.py	screenshotUrl (Cloudinary)
LOOKING_AWAY_LONG	proctor_main.py	screenshotUrl (Cloudinary)
AUDIO_DETECTED	proctor_main.py / Extension	audioLevel (float), screenshotUrl
TAB_SWITCH	Chrome Extension	tabSwitchCount (int)
WINDOW_UNFOCUS ED	Chrome Extension	source (string)

All violation records in Firestore include the following common fields regardless of type: studentId (string), studentName (string), type (string — one of the codes above), timestamp (Firestore SERVER_TIMESTAMP), and violationCount (integer — the running total at the time of this violation).